

NLP

Week 12 – Text Based Features

CS 457 - L1 Data Science

Zeesham Rasheed

- For building any natural language model, the key challenge is how to convert the text data into numerical data.
- As the machine learning models do not understand the text data.
- The idea is to apply some techniques to represent text data into numerical data.

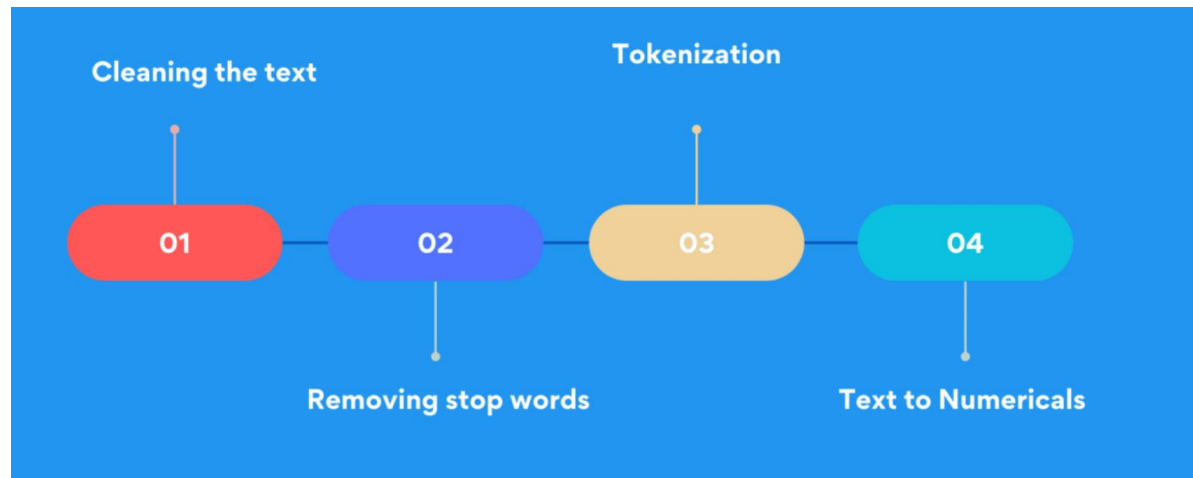
Standard Features



- Here is the standard features we are familiar with.
- Column represents features and rows represents records.
- Used in Regression, Classification and Clustering algorithms

Name ▼	Age ▼	Employment Status ▼	Income ▼	Savings ▼	Credit Score ▼	Credit ▼
John	35	Self-Employed	4500	750	650	YES
Megan	29	Salaried	50000	3000	600	NO
Nick	45	Unemployed	30000	0	650	NO
Mary	27	Self-Employed	60000	1300	0	NO
George	32	Salaried	43000	800	715	YES
Joseph	50	Unemployed	0	0	550	NO
Gina	37	Self-Employed	40000	0	660	NO
Christina	40	Salaried	35000	1400	580	NO
Mike	28	Unemployed	21000	0	700	NO
Lisa	43	Salaried	50000	1000	760	YES

- We have understood the need to reduce words as much as possible, and we do it by text preprocessing techniques like stemming, stop words, etc.
- After this step, we have the step of word embedding. It can be understood as converting a word vector to numbers for better understanding by Machine Learning algorithms.
- There are different word embedding techniques:
 - Bag of Words
 - TF-IDF
 - Word2Vec



- Cleaning the text,
- Removing stop words and punctuation,
- Tokenizing the text into individual words or phrases,
- Converting the text into a numerical representation.

Technique 1: Bag of Words



- The **bag of words** approach represents documents as a vector of word frequencies.
- Although this method can capture the importance of words within a document, it ignores the order of words and fails to encode any semantic information.

Bag of Words - Normalization



- Let's understand this with an example.
- Let's say we have 3 records/documents/corpus/sentences
 - This pasta is very tasty and affordable.
 - This pasta is not tasty and is affordable.
 - This pasta is very very delicious.
- Output of sentences after converting to lowercase (normalization)
 - this pasta is very tasty and affordable.
 - this pasta is not tasty and is affordable.
 - this pasta is very very delicious.

Bag of Words - Tokenization



- Now we will perform tokenization.
 - Dividing sentences into words and creating a list with all unique words and also in alphabetical order.
- We will get the below output after the tokenization step.
 - [“and”, “affordable.”, “delicious.”, “is”, “not”, “pasta”, “tasty”, “this”, “very”]

Bag of Words – Word Frequency

- Creating **Feature Vectors** (features) for each sentence with frequency of words.
- This is called a sparse matrix. Below is the sparse matrix of example sentences.

	and	affordable	delicious	is	not	pasta	tasty	this	very
this pasta is very tasty and affordable.	1	1	0	1	0	1	1	1	1
this pasta is not tasty and is affordable	1	1	0	2	1	1	1	1	0
this pasta is very very delicious.	0	0	1	1	0	1	0	1	2

dataaspirant.com

s.no	and	affordable	delicious	is	not	pasta	tasty	this	very
1	1	1	0	1	0	1	1	1	1
2	1	1	0	2	1	1	1	1	0
3	0	0	1	1	0	1	0	1	2

1-gram bi-gram tri-gram Representation

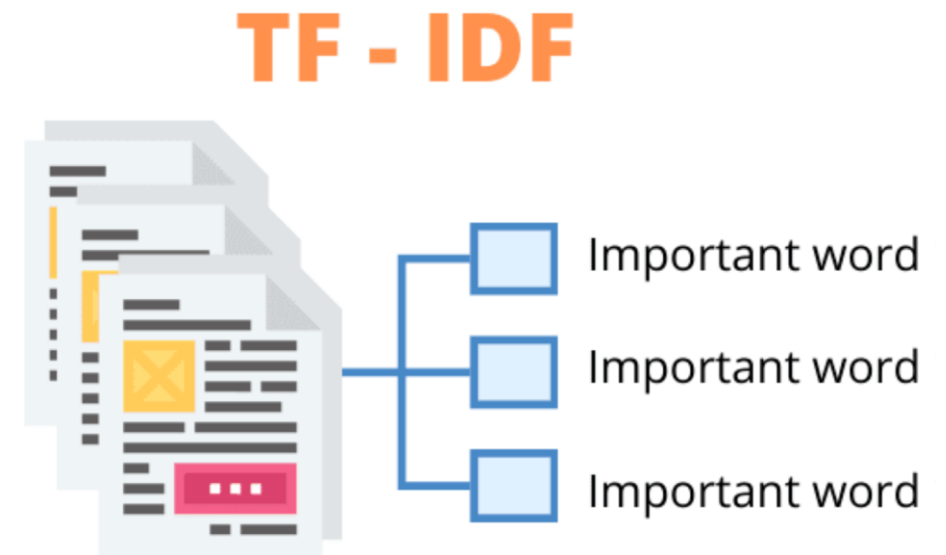


- In the last example we are just taking one word as a feature, another name for this is 1-gram representation, we can also take bi-gram words , tri-gram words etc .
- Examples for Bi-Gram word representation of the first sentence as below.
 - this, pasta
 - pasta, is
 - is, very
 - very, tasty
 - tasty, and
 - and affordable

Technique 2: TF-IDF



- Another popular word embedding technique for extracting features from corpus or vocabulary is TF-IDF.
- **TF-IDF:** Term Frequency – Inverse Document Frequency
- This is a statistical method to find how important a word is to a document (or sentence) all over other documents (sentences)



- **TF**

- TF is Term Frequency (TF). In TF , we are giving some scoring for each word or token based on the frequency of that word.
- The frequency of a word is dependent on the length of the document.
- For Normalization, we will divide the frequency of a word with the length of the document (total number of words)
- Formula to calculate Term Frequency (TF)

$$TF = \frac{\text{\# of times word occurs in a document}}{\text{total number of words in a document}}$$

- **IDF**

- IDF is Inverse Document Frequency.
- Here we assign a score value to a word , this scoring value explains **how much each word is rare and important** across all documents. Rarer words have more IDF score.

- Formula to calculate Inverse Document Frequency (IDF)

- $IDF = \log_e \left(\frac{\text{total \# of documents}}{\text{\# of documents having that word}} \right)$

- Formula to calculate complete TF-IDF value is
 $TFIDF = TF * IDF$

TF-IDF Example 1



- Example sentences :-
 - A: This pasta is very tasty and affordable.
 - B: This pasta is not tasty and is affordable.
 - C: This pasta is very very delicious.

Word	TF			IDF	TF * IDF		
	A	B	C		A	B	C
this	1/7	1/8	1/6	$\log(3/3)=0$	0	0	0
pasta	1/7	1/8	1/6	$\log(3/3)=0$	0	0	0
is	1/7	2/8	1/6	$\log(3/3)=0$	0	0	0
Very	1/7	0	2/6	$\log(3/2)=0.176$	0.025	0	0.058
tasty	1/7	1/8	0	$\log(3/2)=0.176$	0.025	0.022	0
and	1/7	1/8	0	$\log(3/2)=0.176$	0.025	0.022	0
affordable	1/7	1/8	0	$\log(3/2)=0.176$	0.025	0.022	0
not	0	1/8	0	$\log(3/1)=0.477$	0	0.0596	0
delicious	0	0	1/6	$\log(3/1)=0.477$	0	0	0.079

- From the above table we can observe rarer words have more score than common words. That shows us the significance of the words in a sentence/document/corpus.

TF-IDF Example 2



- Input:
 - Document 1: I read the SVM algorithm article in dataaspirant blog
 - Document 2: I read the randomforest algorithm article in dataaspirant blog
- After the text preprocessing (remove stop words) step, we will end up with the below sentences.
 - Document 1: read SVM algorithm article dataaspirant blog
 - Document 2: read randomforest algorithm article dataaspirant blog

TF-IDF Calculation Example							
Words	Count		Term Frequency (TF)		Inverse Document Frequency (IDF)	TF * IDF	
	Document 1	Document 2	Document 1	Document 2		Document 1	Document 2
read	1	1	0.17	0.17	0	0	0
svm	1	0	0.17	0	0.3	0.05	0
algorithm	1	1	0.17	0.17	0	0	0
article	1	1	0.17	0.17	0	0	0
dataaspirant	1	1	0.17	0.17	0	0	0
blog	1	1	0.17	0.17	0	0	0
randomforest	0	1	0	0.17	0.3	0	0.05

Term Frequency and Inverse Document Frequency Calculation

Technique 3: Word2Vec

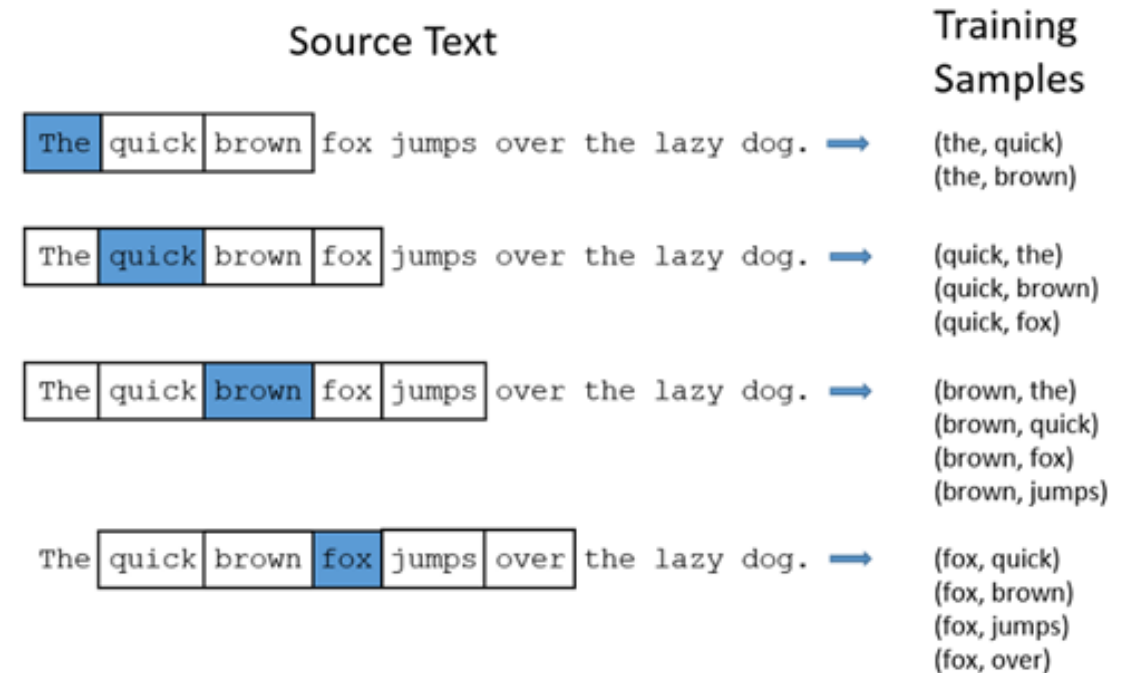


- The Word2Vec model is used for learning vector representations of words (and relationships between words) called “word embeddings”.
 - By using Bag-of-words and TF-IDF techniques we can not capture the meaning or relation of the words from vectors.
- Word2vec constructs such vectors called **word embeddings**.
 - Word embeddings have a capability of capturing semantic and syntactic relationships between words and also the context of words in a document.
 - Intuition is that every word in a sentence is dependent on another word or other words. If you want to find similarities and relations between records, we have to capture word dependencies.
 - Word2Vec was developed by Google researchers, and it is one of the most used embedding methods
 - Word2Vec uses **Deep Learning** techniques for its implementation.

Word2Vec Methods



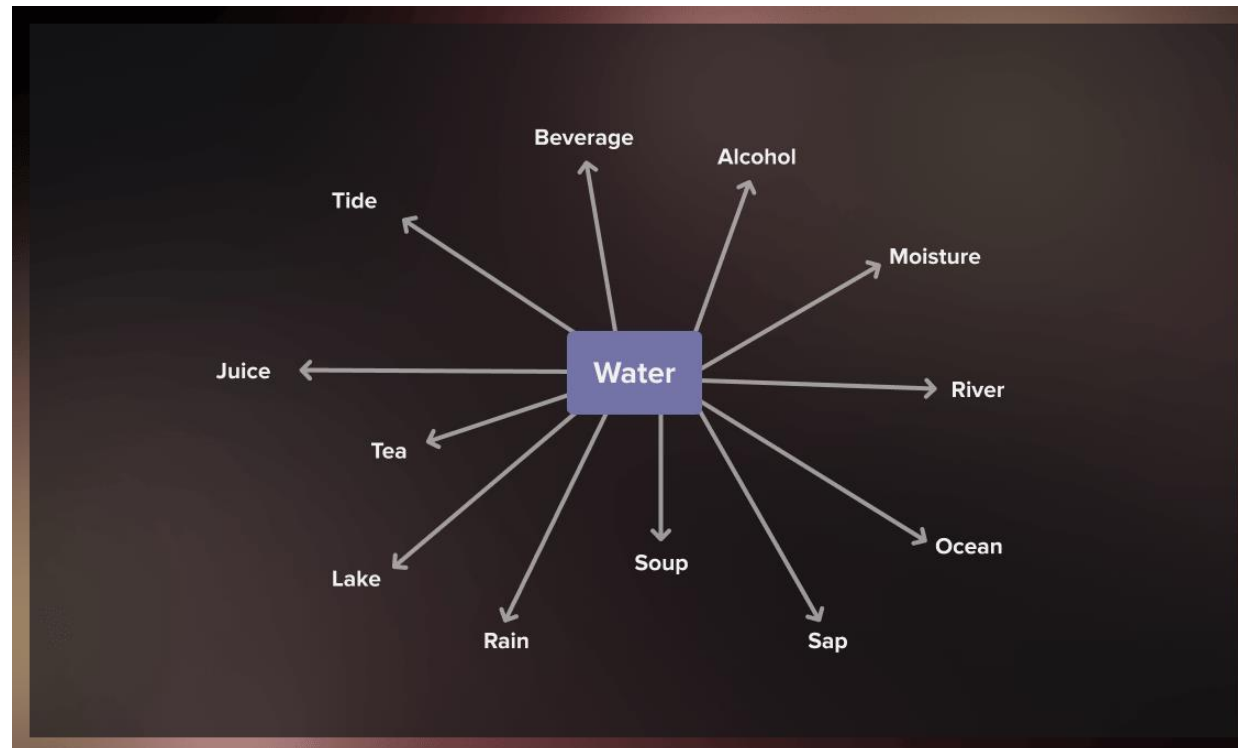
- Word2Vec is a popular neural network-based (deep-learning) word embedding technique that captures semantic relationships between words. It generally uses two methods:
 - Skip-gram
 - CBOW (Continuous Bag of Words)
- Both methods take the center word from the window size words (as an input) and context words (neighbor words) as outputs and learn the relationship between words



Word2Vec – Google Embeddings



- There are many word embedding models available which are already trained using complex neural network (deep learning) techniques on large corpus of data to capture relationships between words



Pretrained Word Embeddings

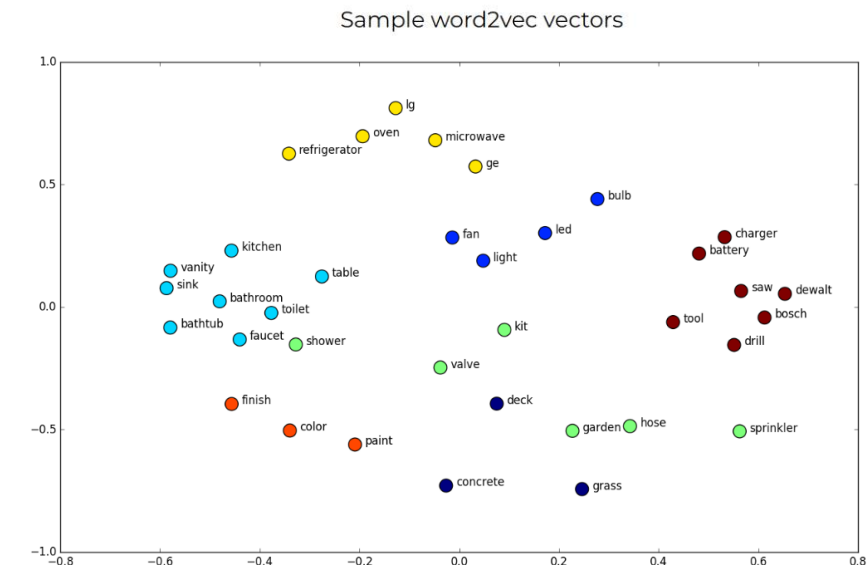
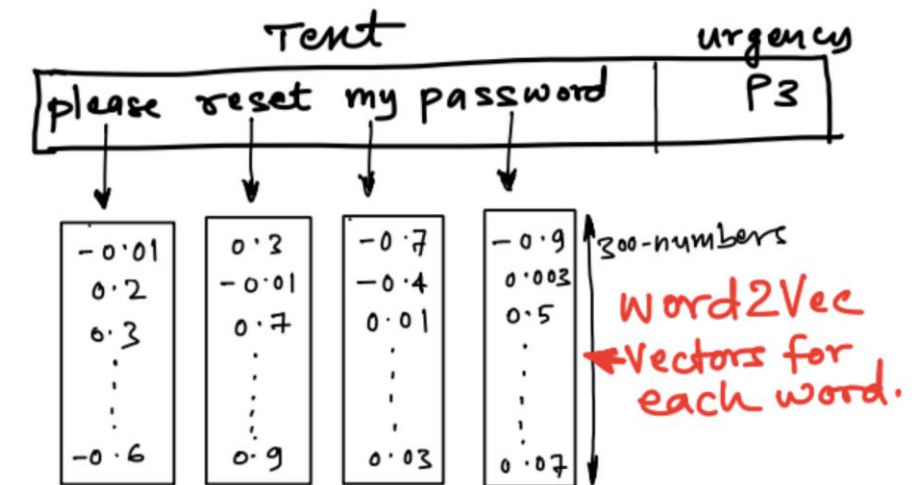


- In many cases, it is not necessary to train word embeddings from scratch. There are several pre-trained word embeddings available that can be used directly in NLP tasks or fine-tuned for specific domains. Some popular pre-trained word embeddings include:
- **Google's Word2Vec:** Pre-trained on the Google News dataset, containing 100 billion words and resulting in a 300-dimensional vector for 3 million words and phrases. (This means that each word is represented by 300 numerical features/columns)
- **Stanford's GloVe:** Pre-trained on the Common Crawl dataset, containing 840 billion words and resulting in 300-dimensional vectors for 2.2 million words.
- **Facebook's FastText:** Pre-trained on Wikipedia, containing 16 billion words and resulting in 300-dimensional vectors for 1 million words.

Word2Vec Representation



- Basically, each word is represented as a vector of numbers.
- Word2Vec vectors are basically a form of word representation that bridges the human understanding of language to that of a machine.
- They have learned representations of text in an n-dimensional space where words that have the same meaning have a similar representation.
 - Meaning that two similar words are represented by almost similar vectors (set of numbers) that are very closely placed in a vector space.



- spaCy
- Gensim
- NLTK
- FastText
- Sentence Transformers

End of Lecture

